

JavaScript Higher Order Functions Review

Callback Functions and the `forEach` Method

- **Definition:** In JavaScript, a callback function is a function that is passed as an argument to another function and is executed after the main function has finished its execution.
- **`forEach()` Method:** This method is used to iterate over each element in an array and perform an operation on each element. The callback function in `forEach` can take up to three arguments: the current element, the index of the current element, and the array that `forEach` was called upon.

```
1  const numbers = [1, 2, 3, 4, 5];
2
3  // Result: 2 4 6 8 10
4  numbers.forEach((number) => {
5    console.log(number * 2);
6  });
```

```
2
4
6
8
10
```

Higher Order Functions

- **Definition:** A higher order function takes one or more functions for the arguments and returns a function or value for the result.

```
1  function operateOnArray(arr, operation) {
2    const result = [];
3    for (let i = 0; i < arr.length; i++) {
4      result.push(operation(arr[i]));
5    }
6    return result;
7  }
8
9  function double(x) {
10   return x * 2;
11 }
```

```
[2, 4, 6, 8, 10]
```

- **map()** Method: This method is used to create a new array by applying a given function to each element of the original array. The callback function can accept up to three arguments: the current element, the index of the current element, and the array that `map` was called upon.

```
1 const numbers = [1, 2, 3, 4, 5];
2 const doubled = numbers.map((num) => num * 2);
3
4 console.log(numbers); // [1, 2, 3, 4, 5]
5 console.log(doubled); // [2, 4, 6, 8, 10]
```

```
[1, 2, 3, 4, 5]
```

```
[2, 4, 6, 8, 10]
```

- **filter()** Method: This method is used to create a new array with elements that pass a specified test, making it useful for selectively extracting items based on criteria. Just like the `map` method, the callback function for the `filter` method accepts the same three

```
[2, 4, 6, 8, 10]
```

- **reduce()** Method: This method is used to process an array and condense it into a single value. This single value can be a number, a string, an object, or even another array. The `reduce()` method works by applying a function to each element in the array, in order, passing the result of each calculation on to the next. This function is often called the reducer function. The reducer function takes two main parameters: an accumulator and the current value. The accumulator is where you store the running result of your operations, and the current value is the array element being processed.

```
1  const numbers = [1, 2, 3, 4, 5];
2  const sum = numbers.reduce(
3    (accumulator, currentValue) => accumulator + currentValue,
4    0
5  );
6
7  console.log(sum); // 15
```

```
15
```

Method Chaining

- **Definition:** Method chaining is a programming technique that allows you to call multiple methods on the same object in a single line of code. This technique can make your code more readable and concise, especially when performing a series of operations on the same object.

```
1  const result = "  Hello, World!  "  
2    .trim()  
3    .toLowerCase()  
4    .replace("world", "JavaScript");  
5  
6  console.log(result); // "hello, JavaScript!"
```

```
"hello, JavaScript!"
```

Working with the `sort` Method

- **Definition:** The `sort` method is used to sort the elements of an array and return a reference to the sorted array. No copy is made in this case because the elements are sorted in place.

```
1  const fruits = ["Banana", "Orange", "Apple", "Mango"];  
2  fruits.sort();  
3  
4  console.log(fruits); // ["Apple", "Banana", "Mango", "Orange"]
```

```
["Apple", "Banana", "Mango", "Orange"]
```

If you need to sort numbers, then you will need to pass in a compare function. The `sort` method converts the elements to strings and then compares their sequences of UTF-16 code unit values. UTF-16 code units are the numeric values that represent the characters in the string. Examples of UTF-16 code units are the numbers 65, 66, and 67 which represent the characters "A", "B", and "C" respectively. So the number 200 appears before the number 3 in an array, because the string "200" comes before the string "3" when comparing their UTF-16 code units.

```
1  const numbers = [414, 200, 5, 10, 3];
2
3  numbers.sort((a, b) => a - b);
4
5  console.log(numbers); // [3, 5, 10, 200, 414]
```

```
[3, 5, 10, 200, 414]
```

The parameters `a` and `b` are the two elements being compared. The compare function should return a negative value if `a` should come before `b`, a positive value if `a` should come after `b`, and zero if `a` and `b` are equal.

```
1  const numbers = [2, 4, 6, 8, 10];
2  const hasAllEvenNumbers = numbers.every((num) => num % 2 === 0);
3
4  console.log(hasAllEvenNumbers); // true
```

true

- **some() Method:** This method checks if at least one element passes the test. The `some( )` method returns `true` as soon as it finds element that passes the test. If no elements pass the test, it returns `false`.

```
1  const numbers = [1, 3, 5, 7, 8, 9];
2  const hasSomeEvenNumbers = numbers.some((num) => num % 2 === 0);
3
4  console.log(hasSomeEvenNumbers); // true
```

true



Assignment

- ☐ Review the JavaScript Higher Order Functions topics and concepts.

Submit

Ask for Help